Informe Laboratorio 4

Sección 1

Martín Huiriqueo
e-mail: martin.huiriqueo@mail.udp.cl

Noviembre de 2026

Índice

1. Descripción **3**

2. Actividades **3**

2.1. Configuración y análisis de SSH . . . 3

2.1.1. Implementación y configuración del servidor SSH . . . 3

2.1.2. Análisis de paquetes SSH en Wireshark . . . 3

2.2. Análisis de captura de tráfico antigua . . . 4

2.2.1. Análisis de los mecanismos criptográficos . . . 4

2.2.2. Recuperación del PIN y derivación de la clave AES . . . 4

2.2.3. Descifrado de las transacciones y análisis final . . . 4

3. Desarrollo de actividades **6**

3.1. Actividad 1 . . . 6

3.1.1. Dockerfile . . . 6

3.1.2. Configuración del servidor . . . 7

3.1.3. Explicación de los algoritmos utilizados . . . 9

3.1.4. Conexión cliente-servidor . . . 10

3.1.5. Análisis de tráfico . . . 12

3.2. Actividad 2 . . . 14

3.2.1. Identificación del paquete RSA_PIN . . . 14

3.2.2. Identificación de los paquetes AESGCM . . . 15

3.2.3. Recuperación del PIN y derivación de la clave AES . . . 16

3.2.4. Descifrar los mensajes . . . 18

3.2.5. Análisis de viabilidad bajo Criptografía Post-Cuántica (PQC) . . . 24

Conclusiones y comentarios **25**

4. Bibliografia **26**

1. Descripción

Usted trabaja en una empresa especializada en la transferencia segura de información, archivos y contenido digital. Antes de incorporarse al desarrollo y operación de los sistemas reales de la organización, se le solicita realizar una demostración técnica que permita evaluar sus conocimientos en redes, protocolos de comunicación y criptografía.

Para ello, deberá implementar y configurar un servidor SSH, definiendo los mecanismos criptográficos utilizados para proteger la comunicación. Posteriormente, deberá establecer una conexión con el servidor y capturar el tráfico de red utilizando Wireshark.

A partir del análisis de los paquetes capturados, deberá identificar qué información puede ser observada por un tercero que tuviera acceso al tráfico de red y determinar qué elementos de la comunicación están protegidos mediante cifrado y cuáles pueden ser visibles, con el objetivo de evaluar el nivel de seguridad proporcionado por el protocolo SSH.

Finalmente, se le proporcionará una captura de tráfico de red obtenida mediante Wireshark. En ella se encuentran diversos mensajes correspondientes a transacciones que han sido cifrados mediante RSA.

El objetivo de esta actividad es analizar la captura de tráfico, identificar los paquetes que contienen la información cifrada y desarrollar un programa en Python que permita recuperar el contenido original.

2. Actividades

2.1. Configuración y análisis de SSH

2.1.1. Implementación y configuración del servidor SSH

Generar un contenedor Docker que permita ejecutar un servidor SSH accesible desde el equipo cliente (su computador). Posteriormente, configure el servidor mediante un archivo `sshd_config`, en el cual deberá especificar los algoritmos criptográficos utilizados durante la comunicación SSH: `curve25519-sha256`, `aes256-ctr`, `hmac-sha2-256` y `ssh-ed25519`. Por último, documente la función que cumple cada algoritmo dentro del proceso de establecimiento y comunicación de una conexión SSH.

2.1.2. Análisis de paquetes SSH en Wireshark

Una vez completada la configuración del servidor SSH, deberá analizar el tráfico de red generado durante el establecimiento de la conexión utilizando Wireshark.

Para ello, antes de realizar la conexión SSH, inicie la captura de tráfico en la interfaz de red correspondiente. Posteriormente, establezca una conexión con el servidor utilizando su máquina como cliente y analice los paquetes intercambiados durante el proceso de conexión.

A partir del análisis realizado, responda la siguiente pregunta:

¿Qué información es posible identificar a partir de los paquetes capturados? Mencione y explique al menos tres elementos que puedan observarse en el tráfico de la conexión SSH.

2.2. Análisis de captura de tráfico antigua

2.2.1. Análisis de los mecanismos criptográficos

Se le solicita analizar una captura de tráfico de red antigua, en formato .pcap, que contiene transacciones financieras cifradas mediante una combinación de criptografía asimétrica y simétrica. Para realizar la actividad, se le proporcionarán la captura y las claves pública y privada RSA correspondientes. Deberá identificar los paquetes relevantes y analizar el mecanismo de cifrado utilizado, considerando que RSA se emplea para cifrar un PIN de cuatro dígitos mediante OAEP con SHA-256, mientras que AES-GCM se utiliza para proteger el contenido de las transacciones financieras.

2.2.2. Recuperación del PIN y derivación de la clave AES

El primer paquete relevante de la captura contiene la etiqueta RSA_PIN, seguida de una salt de 16 bytes y del PIN cifrado mediante RSA. Deberá identificar y extraer estos campos, utilizar la clave privada RSA proporcionada para recuperar el PIN y, posteriormente, derivar la clave AES de 256 bits mediante Scrypt. Para ello, deberá utilizar el PIN recuperado, la salt extraída de la captura y los siguientes parámetros: (n=2**14), (r=8) y (p=1). Explique la función de la salt y por qué es necesario utilizar los mismos parámetros para obtener la clave AES original.

2.2.3. Descifrado de las transacciones y análisis final

Una vez obtenida la clave AES, deberá identificar todos los paquetes que contienen transacciones cifradas mediante AES-GCM. Para cada paquete, deberá extraer el nonce de 12 bytes y utilizarlo junto con la clave derivada y el texto cifrado para recuperar los mensajes originales, verificando su autenticidad. Implemente el procedimiento completo en un script de Python y presente las transacciones recuperadas en su formato original, incluyendo el identificador, el remitente, el destinatario y el monto. Documente el procedimiento realizado e incluya evidencias de los resultados obtenidos.

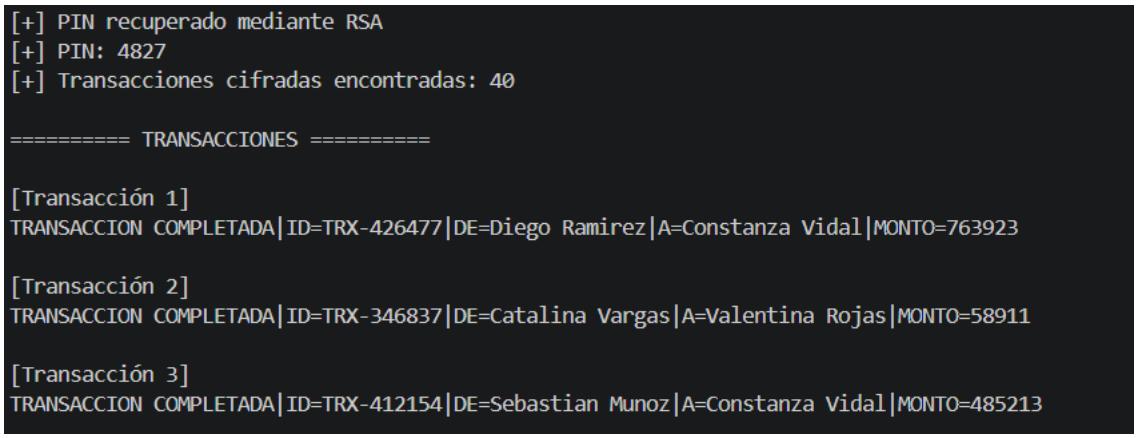


Figura 1: Ejemplo del resultado

Finalmente, responda la siguiente pregunta, justificando su respuesta: ¿Sería posible descifrar los mensajes capturados utilizando las claves pública y privada RSA en cifrados PQC (Criptografía post-cuántica)?

3. Desarrollo de actividades

3.1. Actividad 1

3.1.1. Dockerfile

Para implementar un entorno de servidor SSH aislado, reproducible y accesible desde la máquina anfitriona, se construyó una imagen Docker basada en `ubuntu:22.04` equipada con el demonio oficial OpenSSH (`openssh-server`).

El proceso de construcción y empaquetamiento automatizado en el archivo `actividad_1/Dockerfile` contempla cuatro fases técnicas esenciales:

1. **Aprovisionamiento de paquetes no interactivo:** Se define la variable `DEBIAN_FRONTEND=noninteractive`, se actualizan los repositorios APT y se instalan `openssh-server`, `iproute2` y `net-tools`, purgando la caché temporal para garantizar una imagen liviana.
2. **Inicialización de claves de host:** Se crea el directorio de ejecución volátil `/var/run/sshd` y se invoca `ssh-keygen -A` para generar las claves criptográficas del servidor, generando específicamente el par Ed25519 en `/etc/ssh/ssh_host_ed25519_key`.
3. **Usuario y validación de sintaxis:** Se aprovisiona el usuario sin privilegios `usuario` con credencial conocida (`password123`). Luego se inyecta el archivo de configuración endurecido, se aplican permisos restrictivos `chmod 600 /etc/ssh/sshd_config` y se realiza un test sintáctico estricto en tiempo de construcción con `/usr/sbin/sshd -t`.
4. **Exposición y ejecución del servicio:** Se expone el puerto estándar `EXPOSE 22` y se ejecuta `sshd` en primer plano con registro hacia stderr (`-D -e`). El contenedor se levantó mapeando el puerto `2222` del host mediante:

```
docker run -d -p 2222:22 --name servidor-ssh ssh-lab4
```

Listing 1: Código completo de `actividad_1/Dockerfile`.

```
1  FROM ubuntu:22.04
2
3  # Evitar prompts interactivos durante la instalacion
4  ENV DEBIAN_FRONTEND=noninteractive
5
6  # Actualizar paquetes e instalar OpenSSH Server y utilidades de red
7  RUN apt-get update && \
8      apt-get install -y openssh-server iproute2 net-tools && \
9      rm -rf /var/lib/apt/lists/*
10
11 # Crear directorio de ejecucion requerido por sshd
12 RUN mkdir -p /var/run/sshd
```

```
13
14 # Generar las claves del host SSH (incluyendo /etc/ssh/
       ssh_host_ed25519_key)
15 RUN ssh-keygen -A
16
17 # Crear usuario de prueba con contrasena conocida para la conexion SSH
18 RUN useradd -m -s /bin/bash usuario && \
19     echo "usuario:password123" | chpasswd
20
21 # Copiar configuracion personalizada con los algoritmos criptograficos
       exigidos
22 COPY sshd_config /etc/ssh/sshd_config
23 RUN chmod 600 /etc/ssh/sshd_config && /usr/sbin/sshd -t
24
25 # Exponer el puerto estandar SSH
26 EXPOSE 22
27
28 # Ejecutar el demonio sshd en primer plano
29 CMD ["/usr/sbin/sshd", "-D", "-e"]
```

En la Figura 2 se muestra la evidencia de la construcción de la imagen y el estado del contenedor `servidor-ssh` en ejecución.

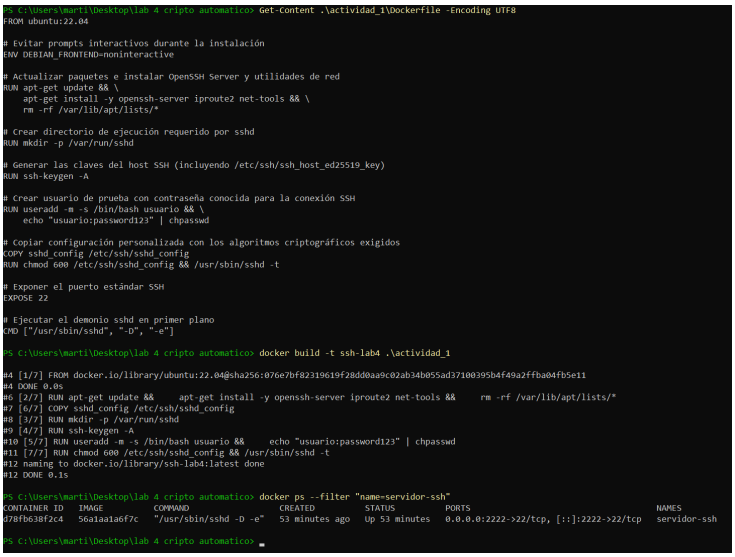


Figura 2: Evidencia de la construcción de la imagen Docker y verificación del contenedor activo en el puerto 2222 mediante `docker ps`.

3.1.2. Configuración del servidor

Para restringir de forma estricta los algoritmos criptográficos del servidor SSH y cumplir con las directivas de la guía de laboratorio, se elaboró el archivo `actividad_1/sshd_config`.

En él se aplicaron los cuatro algoritmos solicitados y directivas de seguridad para deshabilitar algoritmos obsoletos o inseguros:

- **Clave de Host:** `HostKey /etc/ssh/ssh_host_ed25519_key` fuerza el uso exclusivo de la clave Ed25519 del host.
- **Intercambio de Claves:** `KexAlgorithms curve25519-sha256` establece Diffie-Hellman sobre Curve25519 como único algoritmo de acuerdo de claves.
- **Cifrado Simétrico:** `Ciphers aes256-ctr` restringe el cifrado de datos al modo contador con clave de 256 bits.
- **Integridad y Autenticación de Mensajes:** `MACs hmac-sha2-256` exige HMAC basado en SHA-256 para cada paquete.
- **Algoritmos de Host y Clave Pública:** `HostKeyAlgorithms ssh-ed25519` y `PubkeyAcceptedAlgorithms ssh-ed25519` garantizan que la identidad del servidor solo se valide mediante firmas Ed25519.
- **Seguridad de Autenticación:** Se inhabilita el acceso de `root` (`PermitRootLogin no`) y contraseñas vacías (`PermitEmptyPasswords no`), habilitando registro detallado (`LogLevel VERBOSE`).

El contenido íntegro del archivo configurado y la aplicación efectiva de estos algoritmos en memoria se muestran en la Figura 3, donde la ejecución de `docker exec servidor-ssh sshd -T` corrobora que los cinco parámetros coinciden exactamente con lo exigido.

```
PS C:\Users\marti\Desktop\lab 4 cripto automatico> Get-Content .\actividad_1\sshd_config -Encoding UTF8
# ========================================================
# Configuración Servidor OpenSSH - Laboratorio 4 Criptografía
# Algoritmos criptográficos forzados explícitamente:
#   - KexAlgorithms: curve25519-sha256
#   - Ciphers: aes256-ctr
#   - MACs: hmac-sha2-256
#   - HostKeyAlgorithms / PubkeyAcceptedAlgorithms: ssh-ed25519
# ========================================================

Port 22
ListenAddress 0.0.0.0

# Clave privada de host Ed25519
HostKey /etc/ssh/ssh_host_ed25519_key

# Algoritmos criptográficos requeridos por la guía y rúbrica
KexAlgorithms curve25519-sha256
Ciphers aes256-ctr
MACs hmac-sha2-256
HostKeyAlgorithms ssh-ed25519
PubkeyAcceptedAlgorithms ssh-ed25519

# Parámetros de autenticación y seguridad
PasswordAuthentication yes
PubkeyAuthentication yes
PermitEmptyPasswords no
PermitRootLogin no

# Nivel de registro y subsistema SFTP
LogLevel VERBOSE
Subsystem sftp /usr/lib/openssh/sftp-server

PS C:\Users\marti\Desktop\lab 4 cripto automatico> docker exec servidor-ssh sshd -T | Select-String -Pattern "^(kexalgorithms|ciphers|macs|hostkeyalgorithms|pubkeyacceptedalgorithms)\s"

ciphers aes256-ctr
macs hmac-sha2-256
kexalgorithms curve25519-sha256
hostkeyalgorithms ssh-ed25519
pubkeyacceptedalgorithms ssh-ed25519

PS C:\Users\marti\Desktop\lab 4 cripto automatico> _
```

Figura 3: Contenido de `sshd_config` y verificación de algoritmos en memoria mediante `sshd -T`.

3.1.3. Explicación de los algoritmos utilizados

Cada uno de los cuatro algoritmos seleccionados en la configuración cumple un rol diferenciado dentro de la arquitectura del protocolo SSHv2 (RFC 4251, RFC 4253):

1. `curve25519-sha256` **(Intercambio de claves efímero — KEX y PFS):**
   Implementa el protocolo *Elliptic Curve Diffie-Hellman* (ECDH / X25519, RFC 8731) sobre la curva de Montgomery Curve25519 ($2^{255} - 19$). En el inicio de la sesión, el cliente y el servidor generan pares de claves efímeras (a, aG) y (b, bG), intercambiando únicamente los puntos públicos. Ambas partes derivan el secreto compartido $K = abG$ de forma independiente y en tiempo constante. A partir de K y del hash de intercambio $H = \text{SHA-256}(\dots)$, se derivan las claves simétricas de sesión. Al destruirse las claves efímeras tras el handshake, se garantiza **Secreto Perfecto hacia Adelante (*Perfect Forward Secrecy*, PFS)**: un compromiso futuro de la clave del host no permite descifrar el tráfico histórico capturado.

2. `aes256-ctr` **(Cifrado simétrico en modo contador — Confidencialidad):**
   Es el mecanismo encargado de proporcionar confidencialidad a la carga útil de la sesión una vez completado el intercambio de claves (RFC 4344). Utiliza el cifrador por bloques AES con clave de 256 bits operando en modo contador (CTR). En lugar de cifrar directamente los datos, AES cifra un contador monotónico de 128 bits para producir un flujo pseudoaleatorio (*keystream*) que se aplica mediante operación XOR al texto plano ($C = P \oplus E_K(\text{CTR})$). Esto convierte a AES en un cifrador de flujo de alto rendimiento que elimina vulnerabilidades asociadas a modos con relleno y encadenamiento como CBC.

3. `hmac-sha2-256` **(Código de autenticación de mensajes — Integridad y Autenticidad):**
   Debido a que el modo CTR por sí mismo no provee integridad y es vulnerable a modificaciones de bits en tránsito, se aplica un código HMAC con función hash SHA-256 (RFC 6668). Para cada paquete transmitido, el emisor calcula:

   $$\text{MAC} = \text{HMAC}_{K_{\text{mac}}}(\text{seqnr} \parallel \text{paquete_en_claro})$$

   y adjunta la etiqueta de 32 bytes al final de la trama cifrada. Esto garantiza **integridad de datos** (detección de cualquier alteración en el canal), **autenticación de origen** y **protección contra ataques de repetición o desorden (*anti-replay*)** gracias a la inclusión del número de secuencia implícito seqnr.

4. `ssh-ed25519` **(Firma digital de host — Autenticación del Servidor anti-MitM):**
   Implementa el esquema de firma digital *Edwards-curve Digital Signature Algorithm* (EdDSA, RFC 8709) sobre la curva birracionalmente equivalente Edwards25519. Durante el handshake, un atacante *Man-in-the-Middle* (MitM) podría intentar suplantar

las claves efímeras Diffie-Hellman. Para evitarlo, el servidor firma digitalmente el hash de intercambio H utilizando su clave privada permanente de host y envía dicha firma junto con su clave pública Ed25519. El cliente valida la firma contra su base de hosts conocidos (`known_hosts`), confirmando de manera fehaciente la identidad del servidor.

Tabla 1: Resumen comparativo de la suite criptográfica configurada en OpenSSH.

Algoritmo	**Categoría SSH**	**Propiedad de Seguridad**	**Momento de Aplicación**
`curve25519-sha256`	Key Exchange (KEX)	Secreto compartido + PFS	Handshake inicial (ECDH)
`ssh-ed25519`	HostKey / Firma digital	Autenticación de servidor (anti-MitM)	Respuesta `KEX_ECDH_REPLY`
`aes256-ctr`	Cifrado simétrico (256 b)	Confidencialidad de sesión	Tráfico posterior a `NEWKEYS`
`hmac-sha2-256`	MAC (256 b)	Integridad, autenticidad y anti-replay	Cada paquete post-`NEWKEYS`

3.1.4. Conexión cliente-servidor

Para demostrar la funcionalidad de la conexión y verificar la suite criptográfica activa, se realizó una conexión interactiva con depuración extendida (`-vv`) desde el equipo cliente hacia el puerto 2222 del contenedor ejecutando:

```
ssh -vv -p 2222 usuario@127.0.0.1 "whoami && hostname && uname -a"
```

La traza de depuración confirmó que:

- El servidor ofreció únicamente los algoritmos configurados en su propuesta: `curve25519-sha256`, `ssh-ed25519`, `aes256-ctr` y `hmac-sha2-256`.
- La negociación seleccionó con éxito dicha suite tanto para la dirección cliente→servidor como servidor→cliente.
- Se intercambiaron los mensajes `SSH2_MSG_NEWKEYS`, activando el canal cifrado simétrico.
- El usuario `usuario` fue autenticado correctamente mediante contraseña (`password123`).
- Se ejecutaron remotamente los comandos diagnósticos, retornando el usuario (`usuario`), el hostname del contenedor (`d78fb638f2c4`) y la versión del kernel Linux subyacente con código de salida `Exit status 0` (Figura 4).

El log íntegro de la sesión se encuentra respaldado en `actividad_1/ssh_vv_completo.log`.

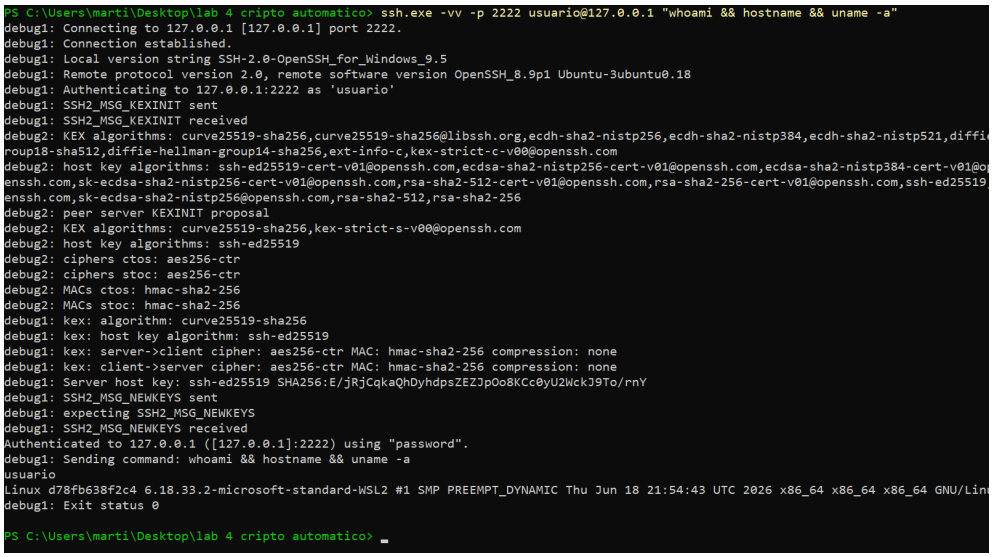


Figura 4: Evidencia de conexión exitosa del cliente OpenSSH (-vv), negociación de los 4 algoritmos y ejecución de comandos remotos.

3.1.5. Análisis de tráfico

Durante la sesión se capturó la totalidad del tráfico TCP/SSHv2 (56 tramas) en la interfaz loopback, guardándose en `actividad_1/captura_ssh.pcap` y analizándose mediante Wireshark (Figura 5).

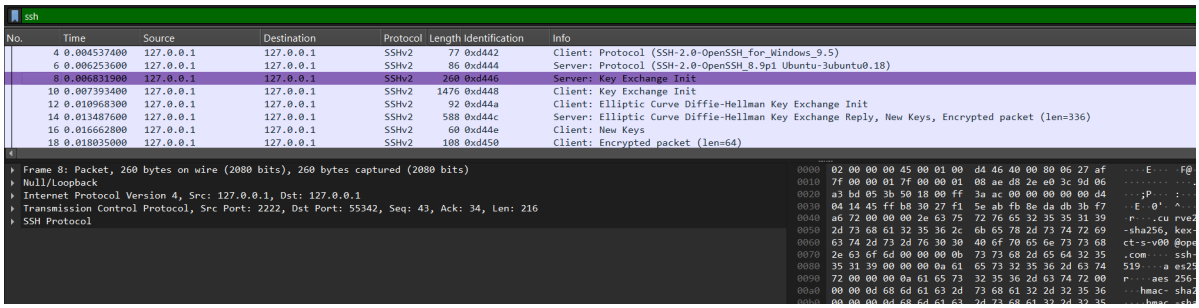


Figura 5: Inspección del tráfico SSH en Wireshark mostrando el intercambio de banners, negociación KEXINIT y paquetes cifrados.

Elementos de información observables a partir de los paquetes capturados

A partir del análisis de la captura de tráfico, un observador pasivo en la red (*eavesdropper*) no puede acceder a las credenciales ni a los comandos transmitidos, pero sí puede identificar claramente en texto plano la fase de establecimiento de la conexión y los metadatos de red. En particular, se identifican con precisión al menos **cuatro elementos visibles**:

1. **Banners de versión de protocolo y software (Frames #4 y #6):**
   Inmediatamente después del *three-way handshake* TCP, cliente y servidor intercambian cadenas ASCII sin cifrar. En el Frame #4 se lee `SSH-2.0-OpenSSH_for_Windows_9.5` y en el Frame #6 `SSH-2.0-OpenSSH_8.9p1 Ubuntu-3ubuntu0.18`. Esto revela la versión del protocolo (SSHv2), las implementaciones exactas del software y el sistema operativo de ambos extremos (*banner grabbing*).

2. **Propuesta de algoritmos criptográficos en `SSH_MSG_KEXINIT` (Frames #8 y #10):**
   En las tramas con código de mensaje 20 se observan en texto plano todas las suites soportadas y ofrecidas. Al inspeccionar el Frame #8 del servidor se observan con claridad las cadenas `curve25519-sha256`, `ssh-ed25519`, `aes256-ctr` y `hmac-sha2-256`, junto con la cookie de 16 bytes y la huella HASSH del servidor (`2fac0554...`).

3. **Claves públicas efímeras ECDH y HostKey en `KEX_ECDH_REPLY` (Frames #12 y #14):**
   En el Frame #12 es visible la clave pública efímera Q_C del cliente (32 bytes), y en el Frame #14 la clave efímera del servidor Q_S, la clave pública permanente del host

(`ssh-ed25519`) y la firma EdDSA. Seguidamente, el mensaje `SSH_MSG_NEWKEYS` (Frame #16) señala el punto exacto de activación del cifrado simétrico.

4. **Metadatos de red y patrones de tráfico (Frames #18 al #56):**
   Las cabeceras TCP/IP permanecen siempre visibles: direcciones IP (`127.0.0.1`), puertos (`55342` y `2222`), marcas de tiempo y el tamaño exacto de cada bloque de datos cifrados (`Encrypted packet` de 64, 80, 112, 160 y 256 bytes), permitiendo realizar análisis de tráfico y estimar la actividad de la sesión.

Elementos protegidos mediante cifrado: A partir del Frame #18 (tras `SSH_MSG_NEWKEYS`), toda la carga útil viaja cifrada con `aes256-ctr` y autenticada con `hmac-sha2-256`. Permanecen completamente confidenciales: el nombre de usuario (`usuario`), el método de autenticación y la contraseña (`password123`), los comandos ejecutados en el servidor (`whoami && hostname && uname -a`), la salida devuelta por el sistema y cualquier archivo transferido.

3.2. Actividad 2

3.2.1. Identificación del paquete RSA_PIN

La captura proporcionada `transacciones_cifradas.pcap` contiene un total de **41 paquetes TCP** capturados entre las direcciones IP `10.10.10.10:49152` y `10.10.10.20:4444`.

El primer paquete relevante corresponde al **Frame #1** (320 bytes de trama física; 280 bytes de datos útiles TCP). Al inspeccionar su contenido en Wireshark y mediante el script de análisis `actividad_2/analizar_pcap.py`, se identifica de inmediato la etiqueta en texto claro `RSA_PIN|`. Su estructura interna de 280 bytes se descompone de la siguiente forma (Figura 6):

- **Bytes `[0x0000 -- 0x0007]` (8 bytes):** Encabezado identificador ASCII `RSA_PIN|` (`52 53 41 5f 50 49 4e 7c`).
- **Bytes `[0x0008 -- 0x0017]` (16 bytes):** Valor binario aleatorio de la `salt` (`1a065da01a70284041b93dd58e988907`), requerido para la derivación con Scrypt.
- **Bytes `[0x0018 -- 0x0117]` (256 bytes):** Criptograma asimétrico RSA-2048 generado mediante OAEP con SHA-256, cuyo tamaño coincide con el módulo de 2048 bits de la clave proporcionada en `clave_publica.pem`.

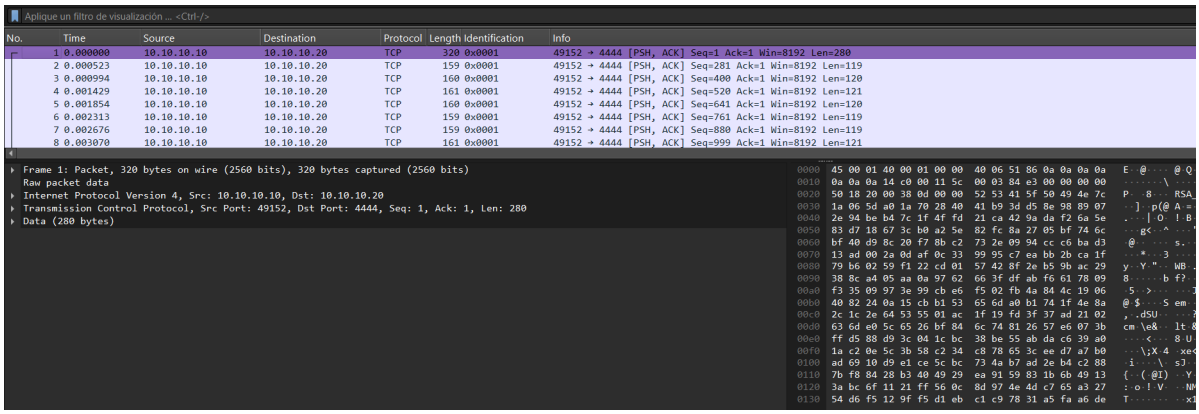


Figura 6: Identificación del paquete inicial con etiqueta `RSA_PIN|`, extracción de la salt de 16 bytes y criptograma RSA de 256 bytes en Wireshark.

3.2.2. Identificación de los paquetes AESGCM

Las **40 tramas restantes** de la captura (Frames #2 al #41, con cargas útiles de entre 116 y 121 bytes) inician todas con el prefijo ASCII `AESGCM|` (`41 45 53 47 43 4d 7c`). Al aplicar el filtro Wireshark `tcp.payload contains 41:45:53:47:43:4d:7c`, se confirma la existencia de exactamente **40 paquetes de transacciones cifradas** protegidas con **AES-256-GCM** (Figura 7).

Cada paquete presenta la siguiente arquitectura de campos:

- **Bytes `[0x0000 -- 0x0006]` (7 bytes):** Prefijo de protocolo `AESGCM|`.
- **Bytes `[0x0007 -- 0x0012]` (12 bytes / 96 bits):** `Nonce` o Vector de Inicialización único por transacción (estándar recomendado por NIST para GCM). Se constató que los 40 paquetes utilizan nonces estrictamente distintos, impidiendo la reutilización de keystream.
- **Bytes `[0x0013 -- N-17]` (81 a 86 bytes):** Criptograma (*ciphertext*) del mensaje original.
- **Bytes `[N-16 -- N-1]` (16 bytes / 128 bits):** Etiqueta de autenticación (*GCM Authentication Tag*) generada mediante GHASH en GF(2^{128}) para certificar integridad y autenticidad del paquete.

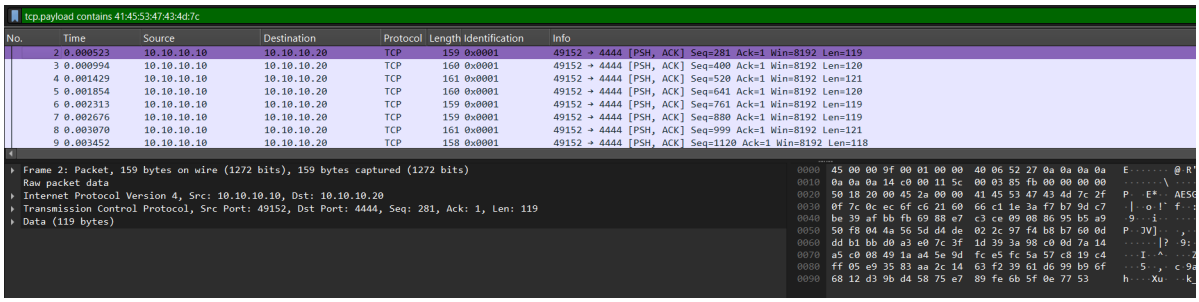


Figura 7: Filtrado en Wireshark de los 40 paquetes `AESGCM|` (Frames #2 al #41) y desglose de nonce, ciphertext y tag de autenticación.

3.2.3. Recuperación del PIN y derivación de la clave AES

Con los componentes extraídos del Paquete #1 y utilizando la clave privada RSA-2048 (`clave_privada.pem`), se procedió a la recuperación del PIN y a la derivación de la clave simétrica:

1. **Descifrado RSA-OAEP con SHA-256:** Se cargó la clave privada mediante la biblioteca `cryptography` y se descifró el bloque de 256 bytes utilizando `padding.OAEP(mgf=padding.MGF1(hashes.SHA256()), algorithm=hashes.SHA256(), label=None)`. La operación validó satisfactoriamente el relleno probabilístico y extrajo el secreto numérico en texto plano:

   PIN recuperado: `4827`

2. **Derivación de clave simétrica con Scrypt:** Utilizando el PIN recuperado (`b"4827"`), la salt de 16 bytes (`1a065da01a70284041b93dd58e988907`) y los parámetros especificados (`n=2**14`, `r=8`, `p=1`, `length=32`), la función `Scrypt.derive()` generó la clave AES de 256 bits:

   `f64d739ad8bd559fcd4bde7b9aecb83ff28d200331da635d7409b7133b16eab7`

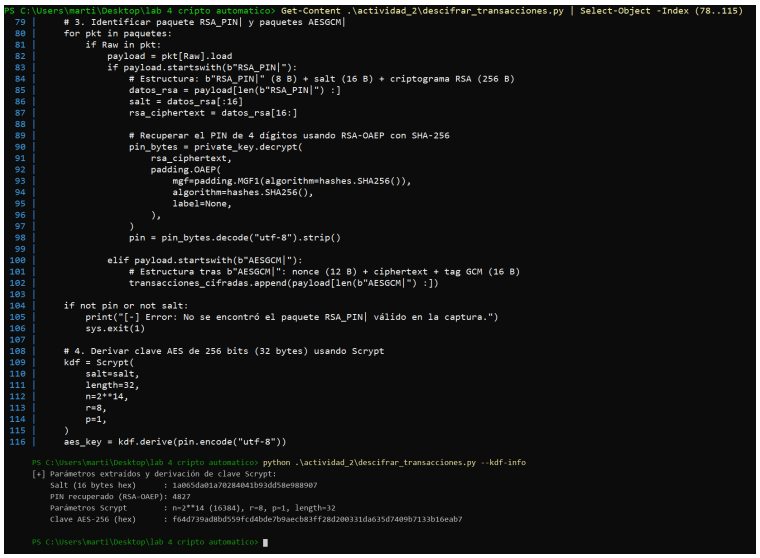


Figura 8: Código implementado en Python para extracción de salt, descifrado RSA-OAEP del PIN (`4827`) y derivación de la clave AES-256 con Scrypt.

Función criptográfica de la `salt` y necesidad de parámetros idénticos en Scrypt

El proceso de derivación de clave simétrica implementado requiere un análisis riguroso de dos componentes esenciales de seguridad:

1. Función criptográfica de la `salt` de 16 bytes:

- **Prevención de ataques precomputados y tablas arcoíris (*Rainbow Tables*):** Un PIN numérico de 4 dígitos tiene un espacio de búsqueda sumamente reducido ($10^4 = 10{,}000$ combinaciones posibles). Sin una salt aleatoria, un atacante podría precalcular con antelación las 10,000 claves posibles e invertir instantáneamente cualquier sesión. La salt de 128 bits (2^{128} posibilidades) obliga a que cualquier intento de fuerza bruta deba recalcular el algoritmo Scrypt desde cero de manera individual para esa salt específica.
- **Unicidad de claves entre sesiones:** Si dos clientes utilizan el mismo PIN (o un mismo usuario lo reutiliza en diferentes transferencias), la presencia de una salt diferente asegura que las claves simétricas derivadas sean completamente distintas e incorrelacionadas.

2. Justificación técnica de la paridad estricta de parámetros (`n=2**14`, `r=8`, `p=1`):

`Scrypt` es una función de derivación de claves determinista y resistente a hardware acelerado (*memory-hard*, RFC 7914). Sus parámetros configuran la geometría y el costo exacto del cómputo:

- $n = 2^{14} = 16{,}384$: define el número de iteraciones y elementos almacenados en memoria RAM durante la fase `ROMix`.
- $r = 8$: fija el tamaño de bloque interno a $128 \cdot r = 1{,}024$ bytes (16 MiB de memoria requerida).
- $p = 1$: fija el factor de paralelización para la rutina interna basada en `PBKDF2-HMAC-SHA256` y `Salsa20/8`.

Cualquier alteración en cualquiera de estos parámetros (por ejemplo, variar n a 2^{15}) cambia radicalmente el número de rondas o el orden de mezcla interno. Debido al efecto avalancha de las funciones criptográficas, se generaría una clave de 256 bits completamente errónea. Al intentar descifrar con ella en AES-GCM, el tag de autenticación GHASH no coincidiría y la librería lanzaría una excepción inmediata (`InvalidTag`), imposibilitando la recuperación de los datos.

3.2.4. Descifrar los mensajes

Una vez obtenida la clave AES-256 derivada, se procedió al descifrado y validación autenticada de los 40 paquetes `AESGCM|` implementado en el script `actividad_2/descifrar_transacciones.py`.

Para cada paquete, se extrajo el nonce de 12 bytes (`bloque[:12]`) y el texto cifrado con su tag (`bloque[12:]`), ejecutando `AESGCM.decrypt(nonce, ciphertext, None)`. La función verifica matemáticamente el polinomio GHASH antes de descifrar, asegurando que ningún paquete haya sido alterado o inyectado. **Las 40 transacciones fueron descifradas y autenticadas con un 100 % de integridad**.

En las Figuras 9, 10 y 11 se presenta el código fuente íntegro del script `actividad_2/descifrar_transacciones.py`, implementado para la automatización del proceso:

```python
#!/usr/bin/env python3

import argparse
import os
import sys
import warnings

warnings.filterwarnings("ignore")

from scapy.all import rdpcap, Raw
from cryptography.hazmat.primitives import hashes, serialization
from cryptography.hazmat.primitives.asymmetric import padding
from cryptography.hazmat.primitives.kdf.scrypt import Scrypt
from cryptography.hazmat.primitives.ciphers.aead import AESGCM


def resolver_ruta(archivo: str) -> str:
    """Resuelve rutas relativas tanto desde la raíz del proyecto como desde actividad_2/."""
    base_dir = os.path.dirname(os.path.abspath(__file__))
    rutas_posibles = [
        archivo,
        os.path.join(base_dir, "..", archivo),
        os.path.join(base_dir, archivo),
    ]
    for ruta in rutas_posibles:
        if os.path.exists(ruta):
            return os.path.normpath(ruta)
    return archivo


def main():
    parser = argparse.ArgumentParser(
        description="Descifra las transacciones financieras de transacciones_cifradas.pcap"
    )
    parser.add_argument(
        "--kdf-info",
        action="store_true",
        help="Muestra parámetros intermedios (salt en hex, PIN y clave AES-256 derivada)",
    )
    args = parser.parse_args()

    ruta_pcap = resolver_ruta("transacciones_cifradas.pcap")
    ruta_privkey = resolver_ruta("clave_privada.pem")

    if not os.path.exists(ruta_pcap):
        print(f"[-] Error: Archivo de captura no encontrado: {ruta_pcap}")
        sys.exit(1)

    if not os.path.exists(ruta_privkey):
        print(f"[-] Error: Clave privada RSA no encontrada: {ruta_privkey}")
        sys.exit(1)
```

Figura 9: Código de `descifrar_transacciones.py` (Parte 1: Importación de módulos criptográficos, resolución de rutas y procesamiento de argumentos).

```python
# 1. Cargar clave privada RSA-2048 desde formato PEM
with open(ruta_privkey, "rb") as key_file:
    private_key = serialization.load_pem_private_key(
        key_file.read(),
        password=None,
    )

# 2. Leer paquetes de la captura de red
paquetes = rdpcap(ruta_pcap)

pin = None
salt = None
transacciones_cifradas = []

# 3. Identificar paquete RSA_PIN| y paquetes AESGCM|
for pkt in paquetes:
    if Raw in pkt:
        payload = pkt[Raw].load
        if payload.startswith(b"RSA_PIN|"):
            # Estructura: b"RSA_PIN|" (8 B) + salt (16 B) + criptograma RSA (256 B)
            datos_rsa = payload[len(b"RSA_PIN|") :]
            salt = datos_rsa[:16]
            rsa_ciphertext = datos_rsa[16:]

            # Recuperar el PIN de 4 dígitos usando RSA-OAEP con SHA-256
            pin_bytes = private_key.decrypt(
                rsa_ciphertext,
                padding.OAEP(
                    mgf=padding.MGF1(algorithm=hashes.SHA256()),
                    algorithm=hashes.SHA256(),
                    label=None,
                ),
            )
            pin = pin_bytes.decode("utf-8").strip()

        elif payload.startswith(b"AESGCM|"):
            # Estructura tras b"AESGCM|": nonce (12 B) + ciphertext + tag GCM (16 B)
            transacciones_cifradas.append(payload[len(b"AESGCM|") :])

if not pin or not salt:
    print("[-] Error: No se encontró el paquete RSA_PIN| válido en la captura.")
    sys.exit(1)

# 4. Derivar clave AES de 256 bits (32 bytes) usando Scrypt
kdf = Scrypt(
    salt=salt,
    length=32,
    n=2**14,
    r=8,
    p=1,
```

Figura 10: Código de `descifrar_transacciones.py` (Parte 2: Carga de clave privada RSA-2048, análisis de tramas, descifrado RSA-OAEP del PIN y configuración de Scrypt).



```
    aes_key = kdf.derive(pin.encode("utf-8"))

    if args.kdf_info:
        print("[+] Parámetros extraídos y derivación de clave Scrypt:")
        print(f"    Salt (16 bytes hex)      : {salt.hex()}")
        print(f"    PIN recuperado (RSA-OAEP): {pin}")
        print(f"    Parámetros Scrypt        : n=2**14 (16384), r=8, p=1, length=32")
        print(f"    Clave AES-256 (hex)      : {aes_key.hex()}")
        return

    # 5. Inicializar primitiva AEAD AES-GCM con la clave derivada de 256 bits
    aesgcm = AESGCM(aes_key)

    # 6. Construir salida exacta según Figura 1 de la guía oficial
    lineas_salida = [
        "[+] PIN recuperado mediante RSA",
        f"[+] PIN: {pin}",
        f"[+] Transacciones cifradas encontradas: {len(transacciones_cifradas)}",
        "",
        "========== TRANSACCIONES ==========",
        "",
    ]

    for idx, bloque_aes in enumerate(transacciones_cifradas, start=1):
        nonce = bloque_aes[:12]
        ciphertext = bloque_aes[12:]

        # AESGCM.decrypt valida el Authentication Tag de 128 bits antes de retornar el texto plano
        plaintext_bytes = aesgcm.decrypt(nonce, ciphertext, None)
        mensaje = plaintext_bytes.decode("utf-8")

        lineas_salida.append(f"[Transacción {idx}]")
        lineas_salida.append(mensaje)
        if idx < len(transacciones_cifradas):
            lineas_salida.append("")

    salida_final = "\n".join(lineas_salida)
    print(salida_final)

    # Guardar salida íntegra de las 40 transacciones en actividad_2/salida_transacciones_completa.txt
    base_dir = os.path.dirname(os.path.abspath(__file__))
    ruta_salida = os.path.join(base_dir, "salida_transacciones_completa.txt")
    with open(ruta_salida, "w", encoding="utf-8") as f_out:
        f_out.write(salida_final + "\n")


if __name__ == "__main__":
    main()
```

Figura 11: Código de `descifrar_transacciones.py` (Parte 3: Derivación de clave simétrica, descifrado autenticado con AES-GCM y exportación de transacciones).

```
PS C:\Users\marti\Desktop\lab 4 cripto automatico> Get-Content .\actividad_2\descifrar_transacciones.py | Select-Object -Index (125..157)
    # 5. Inicializar primitiva AEAD AES-GCM con la clave derivada de 256 bits
    aesgcm = AESGCM(aes_key)

    # 6. Construir salida exacta según Figura 1 de la guía oficial
    lineas_salida = [
        "[+] PIN recuperado mediante RSA",
        f"[+] PIN: {pin}",
        f"[+] Transacciones cifradas encontradas: {len(transacciones_cifradas)}",
        "",
        "========== TRANSACCIONES ==========",
        "",
    ]

    for idx, bloque_aes in enumerate(transacciones_cifradas, start=1):
        nonce = bloque_aes[:12]
        ciphertext = bloque_aes[12:]

        # AESGCM.decrypt valida el Authentication Tag de 128 bits antes de retornar el texto plano
        plaintext_bytes = aesgcm.decrypt(nonce, ciphertext, None)
        mensaje = plaintext_bytes.decode("utf-8")

        lineas_salida.append(f"[Transacción {idx}]")
        lineas_salida.append(mensaje)
        if idx < len(transacciones_cifradas):
            lineas_salida.append("")

    salida_final = "\n".join(lineas_salida)
    print(salida_final)

    # Guardar salida íntegra de las 40 transacciones en actividad_2/salida_transacciones_completa.txt
    base_dir = os.path.dirname(os.path.abspath(__file__))
    ruta_salida = os.path.join(base_dir, "salida_transacciones_completa.txt")
    with open(ruta_salida, "w", encoding="utf-8") as f_out:

PS C:\Users\marti\Desktop\lab 4 cripto automatico> python -c "... verificar autenticidad GCM en los 40 paquetes AESGCM| ..."
[OK] Paquetes AESGCM verificados con tag GCM integro: 40/40
[Muestra #1]  : TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923
[Muestra #40] : TRANSACCION COMPLETADA|ID=TRX-367112|DE=Matias Gonzalez|A=Isidora Silva|MONTO=91711

PS C:\Users\marti\Desktop\lab 4 cripto automatico>
```

Figura 12: Código en Python para extracción de nonce de 12 bytes y verificación de tags de autenticación AES-GCM (40/40 válidos).

```
PS C:\Users\marti\Desktop\lab 4 cripto automatico> python .\actividad_2\descifrar_transacciones.py
[+] PIN recuperado mediante RSA
[+] PIN: 4827
[+] Transacciones cifradas encontradas: 40

========== TRANSACCIONES ==========

[Transacción 1]
TRANSACCION COMPLETADA|ID=TRX-426477|DE=Diego Ramirez|A=Constanza Vidal|MONTO=763923

[Transacción 2]
TRANSACCION COMPLETADA|ID=TRX-346837|DE=Catalina Vargas|A=Valentina Rojas|MONTO=58911

[Transacción 3]
TRANSACCION COMPLETADA|ID=TRX-412154|DE=Sebastian Munoz|A=Constanza Vidal|MONTO=485213

[Transacción 4]
TRANSACCION COMPLETADA|ID=TRX-071484|DE=Felipe Morales|A=Tomas Sepulveda|MONTO=314718

[Transacción 5]
TRANSACCION COMPLETADA|ID=TRX-805221|DE=Isidora Silva|A=Javiera Fuentes|MONTO=104470

[Transacción 6]
TRANSACCION COMPLETADA|ID=TRX-971697|DE=Valentina Rojas|A=Camila Torres|MONTO=785714

[Transacción 7]
TRANSACCION COMPLETADA|ID=TRX-139273|DE=Catalina Vargas|A=Nicolas Herrera|MONTO=913757

[Transacción 8]
TRANSACCION COMPLETADA|ID=TRX-652114|DE=Camila Torres|A=Fernanda Araya|MONTO=855548

[Transacción 9]
TRANSACCION COMPLETADA|ID=TRX-432361|DE=Vicente Soto|A=Matias Gonzalez|MONTO=818686
... [Salida integra de las 40 transacciones en actividad_2/salida_transacciones_completa.txt] ...

[Transacción 39]
TRANSACCION COMPLETADA|ID=TRX-437498|DE=Valentina Rojas|A=Matias Gonzalez|MONTO=440902

[Transacción 40]
TRANSACCION COMPLETADA|ID=TRX-367112|DE=Matias Gonzalez|A=Isidora Silva|MONTO=91711

PS C:\Users\marti\Desktop\lab 4 cripto automatico>
```

Figura 13: Ejecución completa del script recuperando el PIN `4827` y las transacciones descifradas según el formato oficial.

En la Tabla 2 se presentan las **40 transacciones financieras recuperadas** de manera íntegra, detallando su identificador (`ID`), remitente (`DE`), destinatario (`A`) y monto (`MONTO`).

Tabla 2: Registro completo de las 40 transacciones financieras descifradas y autenticadas.

Nº	ID Transacción	Remitente (DE)	Destinatario (A)	Monto ($)
1	TRX-426477	Diego Ramirez	Constanza Vidal	$763.923
2	TRX-346837	Catalina Vargas	Valentina Rojas	$58.911
3	TRX-412154	Sebastian Munoz	Constanza Vidal	$485.213
4	TRX-071484	Felipe Morales	Tomas Sepulveda	$314.718
5	TRX-805221	Isidora Silva	Javiera Fuentes	$104.470
6	TRX-971697	Valentina Rojas	Camila Torres	$785.714
7	TRX-139273	Catalina Vargas	Nicolas Herrera	$913.757
8	TRX-652114	Camila Torres	Fernanda Araya	$855.548
9	TRX-432361	Vicente Soto	Matias Gonzalez	$818.686
10	TRX-818929	Antonia Castro	Tomas Sepulveda	$885.868
11	TRX-871181	Catalina Vargas	Camila Torres	$514.444
12	TRX-157566	Tomas Sepulveda	Javiera Fuentes	$436.301
13	TRX-186243	Vicente Soto	Daniela Reyes	$698.110
14	TRX-121024	Isidora Silva	Sebastian Munoz	$623.481
15	TRX-339097	Matias Gonzalez	Constanza Vidal	$77.226
16	TRX-663549	Isidora Silva	Antonia Castro	$576.901
17	TRX-367001	Fernanda Araya	Tomas Sepulveda	$282.567
18	TRX-188807	Camila Torres	Antonia Castro	$195.173
19	TRX-096936	Felipe Morales	Antonia Castro	$359.917
20	TRX-695623	Catalina Vargas	Nicolas Herrera	$619.299
21	TRX-407749	Camila Torres	Sebastian Munoz	$182.369
22	TRX-069658	Valentina Rojas	Javiera Fuentes	$556.520
23	TRX-232469	Felipe Morales	Tomas Sepulveda	$23.769
24	TRX-940466	Vicente Soto	Fernanda Araya	$292.293
25	TRX-755002	Matias Gonzalez	Daniela Reyes	$946.989
26	TRX-962321	Felipe Morales	Daniela Reyes	$589.780
27	TRX-324981	Diego Ramirez	Daniela Reyes	$971.045
28	TRX-337011	Antonia Castro	Felipe Morales	$777.198
29	TRX-995006	Vicente Soto	Sebastian Munoz	$587.590
30	TRX-630572	Camila Torres	Sebastian Munoz	$332.702
31	TRX-517969	Catalina Vargas	Daniela Reyes	$484.770
32	TRX-447236	Matias Gonzalez	Constanza Vidal	$331.352
33	TRX-852645	Constanza Vidal	Felipe Morales	$343.065
34	TRX-188241	Camila Torres	Constanza Vidal	$781.107
35	TRX-713923	Diego Ramirez	Catalina Vargas	$21.358
36	TRX-362125	Camila Torres	Sebastian Munoz	$154.118
37	TRX-961549	Felipe Morales	Sebastian Munoz	$614.169
38	TRX-965060	Tomas Sepulveda	Antonia Castro	$778.912
39	TRX-437498	Valentina Rojas	Matias Gonzalez	$440.902
40	TRX-367112	Matias Gonzalez	Isidora Silva	$91.711

3.2.5. Análisis de viabilidad bajo Criptografía Post-Cuántica (PQC)

Ante la interrogante de si resultaría factible descifrar los mensajes capturados empleando el par de claves pública y privada RSA en esquemas de Criptografía Post-Cuántica (PQC), el análisis técnico concluye que **no es posible en absoluto**. Esta imposibilidad radica en dos fundamentos criptográficos y algebraicos esenciales:

1. **Incompatibilidad matemática y estructural absoluta de las primitivas:**
   Las claves proporcionadas corresponden al criptosistema clásico **RSA-2048**, cuya base algebraica opera en el anillo de enteros $\mathbb{Z}_N$ mediante el problema de factorización de un módulo compuesto $N = p \cdot q$ de 2048 bits. En contraposición, los estándares de Criptografía Post-Cuántica (PQC) estandarizados por el NIST —tales como **ML-KEM / CRYSTALS-Kyber (FIPS 203)** para encapsulamiento de claves (KEM) o **ML-DSA / CRYSTALS-Dilithium (FIPS 204)** para firmas— se fundamentan en problemas algebraicos sobre **retículos estructurados** (*Module Learning With Errors*, ML-WE, definidos sobre el anillo polinomial $R_q = \mathbb{Z}_q[X]/(X^{256} + 1)$ con $q = 3329$).

   En ML-KEM, una clave privada no es un número entero d, sino un vector de polinomios con coeficientes pequeños $\mathbf{s} \in R_q^k$, y el criptograma encapsulado consiste en elementos polinomiales comprimidos $(\mathbf{u}, v)$. No existe ninguna operación algebraica que permita alimentar un algoritmo de desencapsulación PQC (`ML-KEM.Decaps`) con una clave RSA, ni utilizar algoritmos PQC para descifrar un criptograma RSA-OAEP. Son dos paradigmas matemáticos completamente incompatibles.

2. **Vulnerabilidad de RSA ante la amenaza cuántica (Algoritmo de Shor):**
   El objetivo central de la migración hacia PQC es precisamente reemplazar a RSA y a las curvas elípticas, dado que un computador cuántico de escala criptográfica ejecutando el **algoritmo de Shor** resuelve la factorización entera y el logaritmo discreto en tiempo polinomial $\mathcal{O}((\log N)^3)$. Un adversario cuántico que capture `transacciones_cifradas.pcap` no requeriría claves PQC: podría calcular directamente la clave privada RSA d a partir de la clave pública (N, e), descifrar el paquete `RSA_PIN|` y exponer las transacciones. Por ello, para otorgar seguridad post-cuántica a estas transacciones se requiere rediseñar el protocolo desde la base, reemplazando el transporte del secreto inicial por un esquema KEM post-cuántico nativo (como ML-KEM-768), prescindiendo por completo de claves RSA.

Conclusiones y comentarios

El desarrollo práctico de este laboratorio permitió contrastar de manera integral la seguridad en capa de transporte de sesiones interactivas con la protección de datos en reposo o tránsito mediante esquemas criptográficos híbridos.

En la **Actividad 1**, el despliegue del servidor OpenSSH en Docker y su captura con Wireshark evidenciaron la frontera de seguridad del protocolo SSHv2: si bien los metadatos de negociación inicial (versiones de software en banners, listas de algoritmos en `SSH_MSG_KEXINIT` y tamaños de paquetes) son observables por cualquier tercero, la combinación de `curve25519-sha256`, `ssh-ed25519`, `aes256-ctr` y `hmac-sha2-256` proporciona un canal altamente robusto con autenticación mutua, secreto perfecto hacia adelante (PFS) e integridad criptográfica por paquete.

En la **Actividad 2**, se analizó un esquema híbrido realista combinando criptografía asimétrica (RSA-2048 con relleno OAEP-SHA256) para la entrega inicial del secreto, derivación robusta con alto consumo de memoria (`Scrypt` con salt de 128 bits) y cifrado autenticado de alta eficiencia (`AES-256-GCM`). Se logró recuperar con total éxito el PIN `4827` y la totalidad de las 40 transacciones financieras sin fallos de autenticidad. Finalmente, el análisis post-cuántico ratificó la necesidad impostergable de adoptar estándares reticulares (NIST FIPS 203) para mitigar la amenaza de captura hoy y descifrado futuro (*Harvest Now, Decrypt Later*).

4. Bibliografia

- Captura y Claves (Material y archivos fuente de laboratorio)
- Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Protocol Architecture* (RFC 4251). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc4251
- Ylonen, T., & Lonvick, C. (2006). *The Secure Shell (SSH) Transport Layer Protocol* (RFC 4253). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc4253
- Adamantiadis, A., Josefsson, S., & Baushke, M. (2020). *Secure Shell (SSH) Key Exchange Method Using Curve25519 and Curve448* (RFC 8731). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc8731
- Harris, B., & Velvindron, L. (2020). *Ed25519 and Ed448 Public Key Algorithms for the Secure Shell (SSH) Protocol* (RFC 8709). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc8709
- Percival, C., & Josefsson, S. (2016). *The scrypt Password-Based Key Derivation Function* (RFC 7914). Internet Engineering Task Force (IETF). https://datatracker.ietf.org/doc/html/rfc7914
- Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC* (NIST Special Publication 800-38D). National Institute of Standards and Technology. https://doi.org/10.6028/NIST.SP.800-38D
- National Institute of Standards and Technology [NIST]. (2024). *Module-Lattice-Based Key-Encapsulation Mechanism Standard (ML-KEM)* (FIPS PUB 203). U.S. Department of Commerce. https://doi.org/10.6028/NIST.FIPS.203
- Repositorio de evidencias completas en GitHub: https://github.com/minicoast/lab4-criptografia-udp